The software/hardware contract

A compiler generates instructions that satisfy the ISA. The microarchitecture need not understand a source-language `if` or function. It must implement data movement, address calculation, exceptions, and memory effects correctly.

MIPS32 provides a traceable model of the interface. ARM and x86 supply comparisons. Reconstruct dataflow and addresses rather than memorize a catalog.

Place C, assembly, and machine code on separate levels. Identify where the program counter and stack pointer become architecturally visible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Notation and assumptions

$R[x]$ is a 32-bit register. $M[a]$ denotes data beginning at byte address a. $I[k:j]$ denotes bits k through j. In target calculations, PC is the current instruction address and $PC_{\text{next}} = PC + 4$.

Examples use traditional pre-Release-6 MIPS32, 32-bit instructions, and one branch or jump delay slot. An explicit `nop` occupies every control-transfer delay slot. `jal` saves $PC + 8$. The model does not include the earlier R2000 load-delay requirement.

If the assembler reorders instructions, use its supported ordering control (for example `.set noreorder`) and inspect pseudo-instruction expansion before relying on these exact delay-slot traces.

Instruction and word load/store addresses are four-byte aligned. Declare endianness when interpreting bytes. These assumptions must not be mixed with Release 6 control-transfer semantics.

ISA version, alignment, and byte order answer different questions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 The ISA contract

Programmer-visible state includes general-purpose registers, the program counter, floating-point registers, traditional HI and LO, and privilege and exception state.

Software may depend on instruction semantics, but not on pipeline depth or cache placement. Timing may change while architectural results remain the same. The ISA also defines exceptional conditions, including overflow and address faults. Privileged specifications determine exception-state and recovery rules.

"This instruction adds" is architectural. "This adder finishes in one cycle" is an implementation requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.2 Arithmetic operations

```
add $t0, $t1, $t2
```

The result is $R[t0] \leftarrow R[t1] + R[t2]$ when representable. `add`, `addi`, and `sub` trap on signed overflow. `addu`, `addiu`, and `subu` retain the modular result without that trap.

The `u` in `addiu` does not make its immediate unsigned. The 16-bit immediate is sign-extended. Choosing the trapping variant changes an observable exception behavior.

Distinguish operand extension from overflow handling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expressiveness and cost

A small set of operations can express complex language behavior through composition. Multiplication can use iteration, shift-and-add, or a multiplier. The presence of `mult` changes available instruction sequences.

Fixed-width encoding gives predictable field positions but limits immediates. A complex instruction can reduce instruction count while adding decode or execution cost. Compare count, work per instruction, encoding, and compiler use together.

A RISC or CISC label alone is not a performance model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 Registers and operands

MIPS32 has 32 general-purpose registers of 32 bits. Scalar values are commonly allocated to registers. Loads and stores move data between registers and memory.

Ordinary arithmetic in this load/store architecture does not take two arbitrary memory locations as operands. Register-file ports and bypass paths are microarchitectural. The ISA specifies the value subsequent execution must observe.

Translate `c = a + b` using resident register values. A source-variable reference does not necessarily cause a load.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Memory operands

$$EA = R[base] + \text{signextend}(imm16) \pmod{2^{32}}.$$

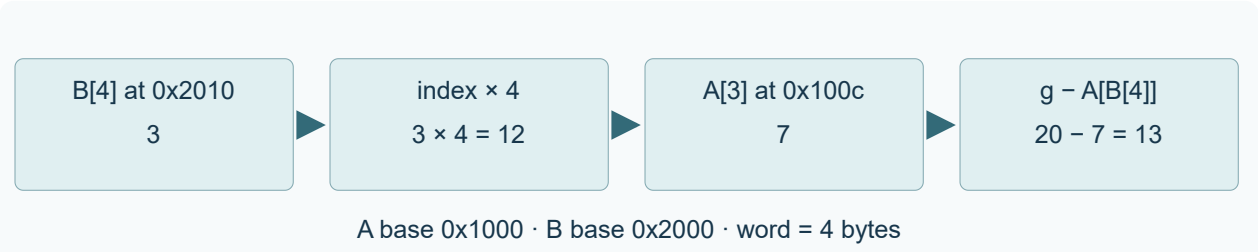
`lw $t0, 12($s0)` loads the word beginning at that effective address. `sw` stores register data there. The signed displacement ranges from −32768 through 32767 bytes.

Large offsets need an address-construction sequence. Address formation does not remove alignment, translation, and permission checks.

Separate the address calculation from the interpretation of loaded data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Indirect indexing: trace both reads



Each array word is 4 bytes; base $A = 0x1000$, $B = 0x2000$, $B[4] = 3$, $A[3] = 7$, and $g = 20$. Assume valid aligned addresses and indexes, and no arithmetic exception.

```
lw   $t0, 16($s7)   # B[4] = 3
sll  $t0, $t0, 2    # 3 × 4 = 12 bytes
addu $t0, $t0, $s6  # address A[3] = 0x100c
lw   $s0, 0($t0)    # A[3] = 7
sub  $s0, $s1, $s0  # f = 20 − 7 = 13
```

Here `$s6=0x1000`, `$s7=0x2000`, and `$s1=20`; 16 is the byte offset for $B[4]$, not the index itself.

Immediate operands

`addi $t0, $t0, -7` sign-extends the immediate. `lui` places its immediate in the upper half and clears the lower half.

```
lui $t0, 0x1234
ori $t0, $t0, 0x5678
```

This constructs `0x12345678`. `ori` zero-extends its immediate. A fixed 32-bit instruction cannot encode an arbitrary 32-bit constant plus all necessary opcode and register fields.

A memory literal may produce the same value with different code-size and cache costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.4 Signed and unsigned integers

Unsigned n-bit values span 0 through $2^n - 1$. Two's complement gives the high bit weight -2^{n-1}, so

$$-2^{n-1} \le x \le 2^{n-1} - 1.$$

The 32-bit range is −2147483648 through 2147483647. Complementing and adding one produces the negative bit pattern. Signed and unsigned addition can use the same adder, with different overflow interpretation. Comparisons and multiplication or division also require the chosen interpretation.

`0xfffffff6` is unsigned 4294967286 or signed −10.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Overflow and comparisons

Same-sign operands producing an opposite-sign result indicate signed addition overflow. For operand signs s_a, s_b and result sign s_r,

$$overflow = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

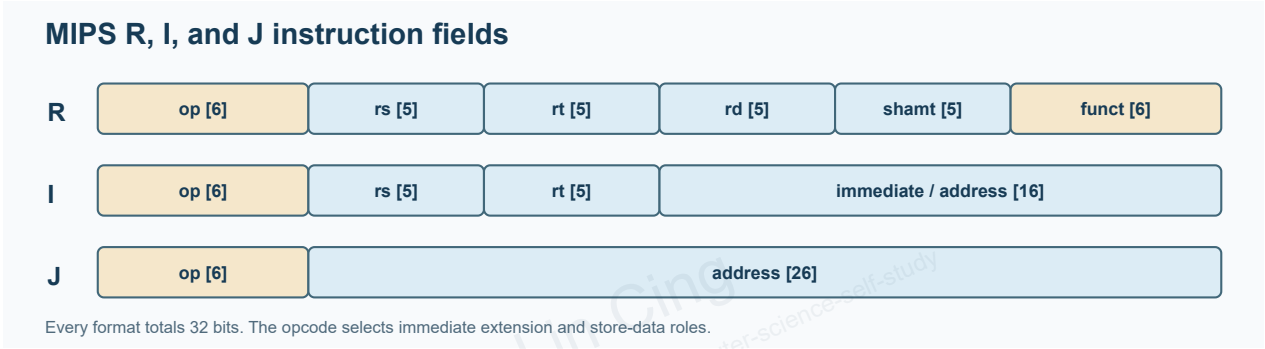
Unsigned overflow is carry out. `slt` and `sltu` implement different orderings of the same bits. `0xffffffff + 1` carries out in unsigned arithmetic, but signed $-1 + 1$ does not overflow.

Interpretation precedes an overflow judgment.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.5 Instruction formats

R-format uses an opcode and function field. I-format contains a 16-bit immediate. J-format contains a 26-bit target index. Each occupies 32 bits.



Predictable register-field positions permit early register reads. Limited immediate width shifts work to multistep compiler-generated sequences.

Bit positions run from MSB 31 to LSB 0. This diagram is not a physical layout.

R, I, and J fields

R-format is `op[31:26]`, `rs[25:21]`, `rt[20:16]`, `rd[15:11]`, `shamt[10:6]`, `funct[5:0]`.

I-format is `op`, `rs`, `rt`, `imm16`. Arithmetic and address displacements are sign-extended. Logical immediates are zero-extended. Loads and stores use the immediate for the address. Store data comes from `rt`.

J-format is `op` and `target26`. It cannot specify every absolute 32-bit target independently of location.

`rt` is a destination for `addi` and a source for `sw`. Its role follows the opcode.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6 Logic and shifts

Bitwise operations include `and`, `or`, `nor`, and `xor`. Logical immediate forms zero-extend their masks. Logical shifts insert zeros. Arithmetic right shift replicates the sign bit. `sll`, `srl`, and `sra` can encode the shift amount in `shamt`.

```
srl  $t1, $t0, 8
andi $t1, $t1, 0x000f
```

This extracts bits 11 through 8. AND clears unselected bits, OR sets bits, and XOR toggles selected bits.

Match mask and shift before interpreting the extracted field.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.7 Decisions and branches

`beq rs, rt, label` chooses the target on equality. In this traditional model, its next instruction executes regardless of branch direction. Execution then reaches the target or continues after the delay slot.

Examples use `nop` in that slot. Release 6 control transfers require their own semantics. Comparison pseudo-instructions may expand into `slt` and a branch.

Branch delay is part of this architectural model. It does not remove data hazards.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Translating a loop

```
int sum = 0;
for (int i = 0; i < n; ++i)
    sum += a[i];
```

Let `$a0` hold the array base, `$a1` hold n, and `$v0` hold the sum.

```
move $v0, $zero
move $t0, $zero
beq  $t0, $a1, done
nop
loop:
sll  $t1, $t0, 2
addu $t1, $t1, $a0
lw   $t2, 0($t1)
addu $v0, $v0, $t2
addiu $t0, $t0, 1
bne  $t0, $a1, loop
nop
done:
```

Translating a loop (continued)

Assume $n \geq 0$, valid aligned storage, and no address wrap. The sum is modular unless inputs guarantee representable signed C arithmetic. Scaling by four converts word index to bytes.

At iteration i, the accumulator equals the sum of the first i elements modulo 2^{32}. Initialization, preservation, and exit prove the result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Switch strategies

Dense case values can use a jump table. Check the index range, scale the index, load the target, and use `jr` with its delay slot. Sparse cases may use a branch chain or a binary decision tree.

Tables reduce decision depth but consume space and add indirect-branch and cache costs. The choice depends on case distribution, safety, and implementation behavior.

Indirect prediction and table locality can outweigh source-code brevity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete decoding and control-flow trace

Decoding must determine format and field roles before interpreting a mnemonic.

Word	Fields	Architectural effect
`0x8fa8000c`	`op=0x23, rs=29, rt=8, imm16=12`	`lw $t0,12($sp)`, with `EA=R[sp]+12`
`0x012a4020`	`op=0, rs=9, rt=10, rd=8, shamt=0, funct=0x20`	`add $t0,$t1,$t2`
`0x0c100140`	`op=3, target26=0x100140`	`jal` to `0x00400500` in the same 256-MiB region

I-format immediates are not uniformly signed. Loads and `addi/addiu` sign-extend; `andi/ori/xori` zero-extend. A store reads its data from `rt`. The opcode determines whether `rt` is a source or destination.

MIPS `addu/addiu` compute modulo 2^{32}. C unsigned arithmetic is also modulo its type width, but C signed overflow is undefined behavior. Signed source programs require a proof that values remain representable, or an unsigned/wider type, before a modulo machine result can be claimed to implement their semantics.

For `0x1109fffc` at `0x00400020`, $imm16 = 0xfffc = -4$:

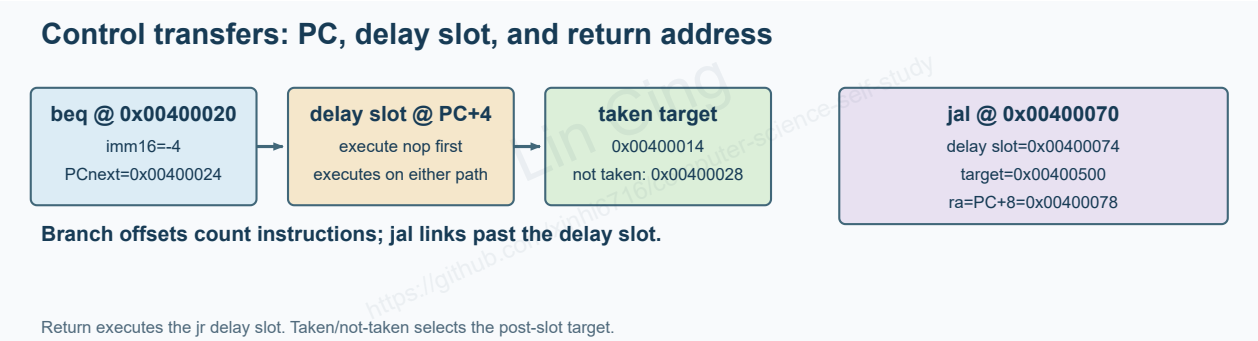
$$PC_{\text{next}} = 0x00400020 + 4 = 0x00400024,$$

Complete decoding and control-flow trace (continued)

$$target = PC_{\text{next}} + (\text{sign_extend}(0xfffc) \ll 2) = 0x00400024 - 16 = 0x00400014.$$

The taken PC sequence is `0x00400020 → 0x00400024 → 0x00400014`. The not-taken sequence is `0x00400020 → 0x00400024 → 0x00400028`. The delay-slot instruction at `0x00400024` executes in both cases.

For a `jal` at `0x00400070`, its slot is `0x00400074`, target `0x00400500`, and `$ra=PC+8=0x00400078`. `jr $ra; nop` executes its own delay slot before reaching that return address.



Instruction addresses must be word aligned. Branch offsets count instructions. J-format targets inherit the high four bits of PC+4. Mixing bytes and words, or using the current PC's high bits at a region boundary, changes the target.

2.8 Procedure calls

`jal target` saves $PC + 8$ in `$ra` and transfers after its delay slot. `jr $ra` returns after its own delay slot. A nested `jal` overwrites the incoming return address, so a nonleaf procedure must preserve that address when it needs a normal return.

The ABI defines calling conventions. Hardware does not automatically allocate the full frame or preserve all registers.

Trace `main` to `f` to `g`. Locate the point where an unsaved continuation is lost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Register responsibilities

Register	Convention
`$zero`	Always zero
`$a0`–`$a3`	Arguments
`$v0`, `$v1`	Results
`$t0`–`$t9`	Caller-saved temporaries
`$s0`–`$s7`	Callee-saved values
`$sp`	Stack pointer
`$ra`	Return address

A caller preserves temporary values it still needs after a call. A callee restores saved registers it modifies. Hardware does not infer these obligations from register names.

Preservation is conditional on modification and liveness, not on a register merely being mentioned.

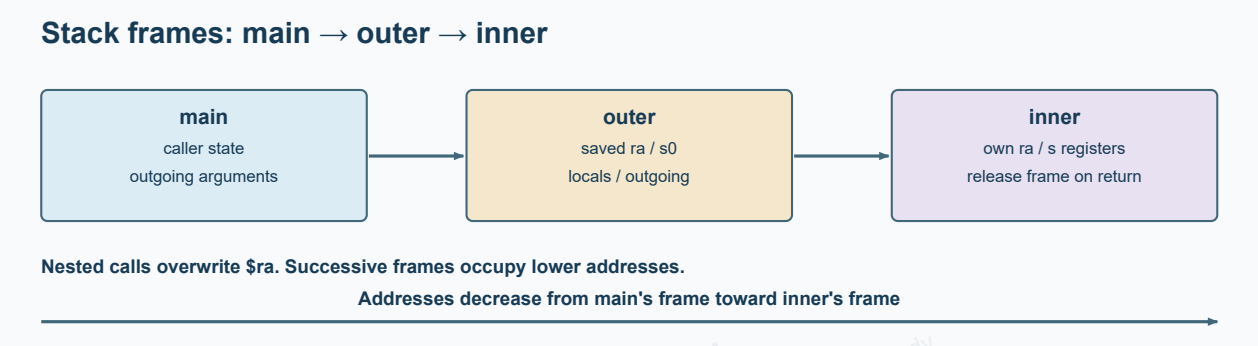
Nested calls and the return address

```
addiu $sp, $sp, -8
sw    $ra, 4($sp)
jal   g
nop
lw    $ra, 4($sp)
addiu $sp, $sp, 8
jr    $ra
nop
```

This conceptual minimal frame preserves the incoming return address. It is not a full ABI template. Real frames may need outgoing arguments, locals, spills, and saved registers with required alignment.

The return-address restore belongs after the call's delay slot, not inside it.

Stack frames



A frame can hold the saved return address, saved registers, local arrays, spills, and outgoing arguments. Simultaneously live slots cannot overlap.

Entry subtracts frame size from `$sp`. Exit restores state and the incoming `$sp`. ABI rules determine slot alignment and argument layout. A pointer into a returned frame does not extend its lifetime. Another call can overwrite that storage.

Mark high and low addresses. These examples use a downward-growing stack.

Heap storage

Stack lifetime follows procedures. Heap objects allocated through `malloc` or `new` persist until deallocation or collection. Allocators manage free blocks, metadata, and fragmentation.

Allocation may fail or trigger page faults and virtual-address-space expansion. Successful virtual allocation does not imply physical residency of every page.

Returning a pointer to a local integer does not preserve that object after return; dereferencing it violates the object lifetime even if the stack bytes have not yet changed. A heap leak instead loses the last usable pointer to a still-allocated object, preventing its release. Repeated allocation followed by overwriting the sole pointer can accumulate leaks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.9 Characters and bytes

ASCII defines seven-bit code points commonly stored in eight-bit bytes. Unicode uses encodings such as UTF-8 and UTF-16. C strings use a zero terminator; other formats may carry length metadata.

`lb` sign-extends a byte. `lbu` zero-extends it. `sb` stores a byte. Copying a string requires adequate destination capacity and correct termination.

Endianness orders multibyte values and must be specified at exchange boundaries.

Bytes `41 42 43 00` form the NUL-terminated ASCII string "ABC". As one 32-bit integer they mean 0x41424300 in big-endian order or 0x00434241 in little-endian order. Bits alone do not determine the type or byte order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.10 Large constants

`lui` sets bits 31 through 16 and clears the lower half. `ori` fills that half without sign extension. Construct `0x8000abcd` using `lui` with `0x8000` and `ori` with `0xabcd`.

`lui` followed by `addiu` uses a signed low half. When low bit 15 is set, the high half must be increased by one to compensate. Appendix B derives the relocation rule.

The high half for a logical OR sequence is not always the correct high half for a signed-low-half addition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Branch and jump targets

$$target = (PC + 4) + (\text{signextend}(imm16) \ll 2).$$

The displacement spans −131072 through 131068 bytes relative to $PC + 4$. Its encoded unit is an instruction word.

$$target = (PC + 4)[31 : 28] \parallel target26 \parallel 00_2.$$

J-format selects the 256-MiB region of $PC + 4$, which can differ from the current instruction's region at a boundary.

Saving $PC + 8$ for a delayed call does not change the branch-target base from $PC + 4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Addressing modes

The examples use register, immediate, base-plus-displacement, PC-relative, and pseudo-direct addressing. `lw $t0, 0($sp)` uses the architectural base value for that instruction. Forwarding or stalls handle dependencies. A branch delay slot does not preserve an old base value.

Extra addressing modes consume encoding space and complicate decode and exceptions. Pseudo-direct targets save bits but restrict placement. A linker may need a trampoline or indirect transfer for an unreachable target.

Equal addresses reached by two modes can depend on different base values.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decoding machine code

```
0x012a4020
op    = 000000
rs    = 01001   $t1
rt    = 01010   $t2
rd    = 01000   $t0
shamt = 00000
funct = 100000  add
```

This is `add $t0, $t1, $t2`. Decode the opcode first. Confirm ISA, register names, and instruction length before field extraction.

Interpreting variable-length x86 bytes as MIPS fields cannot establish a meaningful instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.11 Atomic synchronization

Two cores can load x, independently compute $x + 1$, and both store it. One increment is lost.

`ll` loads and establishes a reservation. `sc rt, offset(base)` uses the old `rt` as store data. Success atomically stores that data and sets `rt` to 1. Failure writes no shared data and sets `rt` to 0.

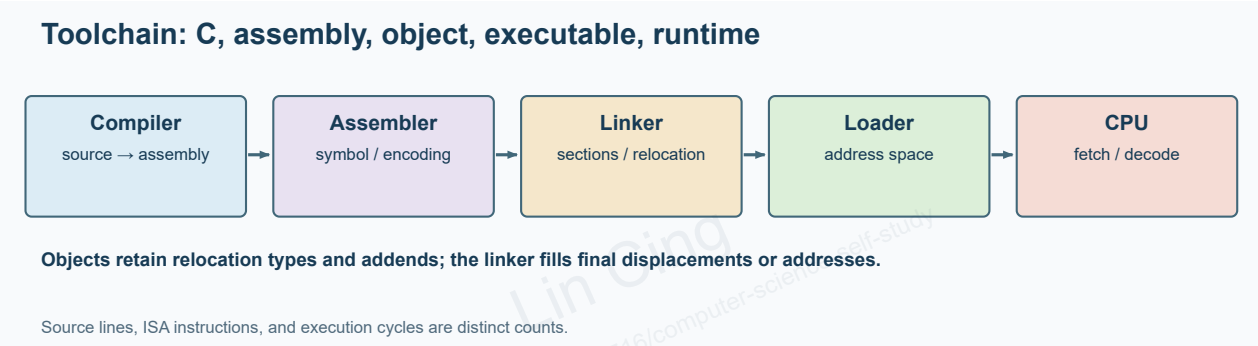
```
retry:
ll    $t0, 0($s0)
addiu $t0, $t0, 1
sc    $t0, 0($s0)
beq   $t0, $zero, retry
nop
```

Retry rereads the counter. Reservation granularity and invalidation events depend on architectural and implementation rules. Neither atomicity nor this loop alone guarantees fairness or ordering of unrelated memory.

An uninitialized different `sc` register would store incorrect data even if its reservation succeeded.

2.12 From C to an executable

The compiler selects instructions and optimizes. The assembler expands pseudo-instructions and emits encodings, symbols, and relocations. The linker combines sections and resolves references. The loader creates mappings and initial state.



Language-semantics errors, invalid instruction operands, undefined symbols, and mapping failures belong to different stages.

Trace an external function through assembly relocation and runtime resolution.

Address meaning in the toolchain

Labels initially identify section-relative locations. External symbols may remain unresolved. Relocations identify the operation needed when final symbol values become known.

Static linking resolves references into an executable. Position-independent shared libraries can share physical code pages while allowing runtime resolution.

Virtual addresses, file offsets, segment-relative offsets, and physical addresses are distinct coordinates. Loader mappings connect them but do not make them identical.

A runtime relocation can change an address without moving the corresponding bytes within the file.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A calling-convention example

```
int twice_sum(int x, int y) {
    int s = x + y;
    return s * 2;
}
```

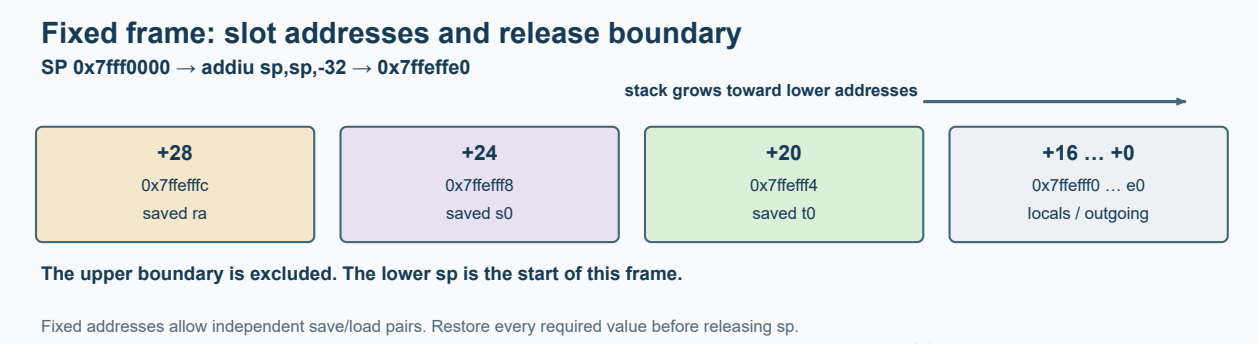
For inputs whose signed operations are representable,

```
twice_sum:
addu $v0, $a0, $a1
sll  $v0, $v0, 1
jr   $ra
nop
```

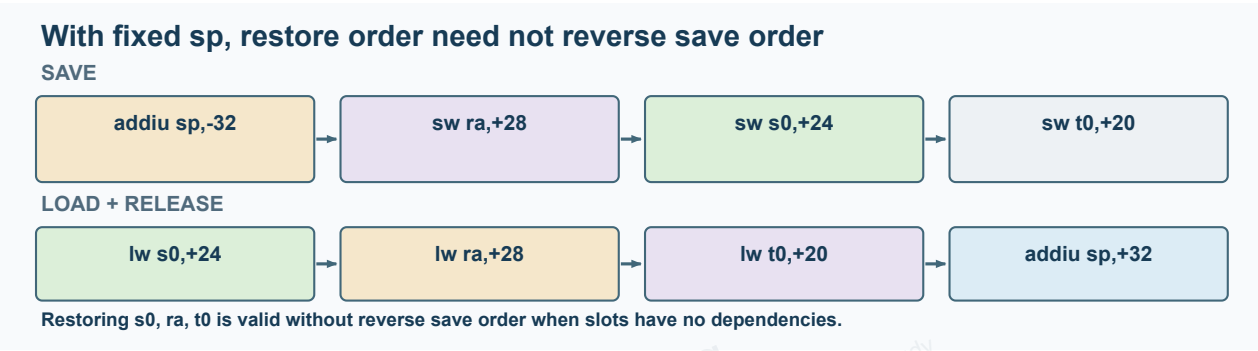
The leaf makes no nested call and modifies no saved register, so it need not preserve `$ra`. Outside the valid signed-input range, modular machine behavior does not establish defined signed C behavior.

Use of `$s0` would add a restoration obligation.

Fixed-frame slot addresses



Fixed-frame save, load, and release order



Entry `sp=0x7fff0000` and a 32-byte allocation give `sp=0x7ffeffe0`.

Slot	Address	Saved value	Restored value
`28(sp)`	`0x7ffefffc`	`ra`	`ra`
`24(sp)`	`0x7ffefff8`	`s0`	`s0`
`20(sp)`	`0x7ffefff4`	`t0`	`t0`

Saving `ra,s0,t0`, then restoring `s0,ra,t0`, then adding 32 to `sp` is valid. Restore order need not reverse save order. The base stays fixed, the slots are distinct, and load destinations have no mutual dependencies.

Fixed-frame save, load, and release order (continued)

Reverse order is necessary for a push/pop sequence that changes `sp`, or when slot/register dependencies require it. Release timing remains mandatory. Load all required values before increasing `sp`; an old offset applied to the new base addresses a different location.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursive activation trace

`main` calls `f` at `0x00400010`, producing return address `0x00400018`. With entry `sp=0x7fff0000`, `f` allocates 32 bytes and reaches `sp=0x7ffeffe0`.

Store	Address	Value
`sw $ra,28($sp)`	`0x7ffefffc`	`0x00400018`
`sw $s0,24($sp)`	`0x7ffefff8`	caller's old `$s0`

A recursive `jal f` at `0x00400120` sets `$ra=0x00400128`. The inner activation allocates its own frame at `sp=0x7ffeffc0` and saves that return address at `0x7ffeffdc`. Both frames are live. `sp+28` always identifies the current activation's slot; the inner base cannot read the outer slot with the same offset.

The base case restores its own `$ra` and `$s0`, releases its frame to `0x7ffeffe0`, and returns to `0x00400128`. The outer activation eventually reloads `0x00400018` from `0x7ffefffc`, releases 32 bytes, and returns to `main`. Every `jal` and `jr` retains its `nop` delay slot. Removing it changes the executed sequence.

This trace establishes the need to save `$ra` before a nested call, compute slots from the current base, and complete restores before release.

Arrays and pointers

$$address = base + i \times element_size.$$

Pointer traversal increments by element size. It can avoid repeated scaling, but the compiler can apply the same strength reduction to an indexed loop.

Pointer width is platform-dependent. This machine model uses 32-bit addresses. C and C++ object rules constrain pointer arithmetic. A hardware-generated bit pattern need not be a valid language-level dereference.

Compare generated instructions rather than source spelling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete sorting dataflow and cost

Insertion by adjacent swaps connects C control flow, array addresses, and procedure calls:

```
for (int i = 0; i < n; i += 1)
  for (int j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1)
    swap(v, j);
```

For `v=[3,1,2]`, `i=0` has no inner iteration. At `i=1,j=0`, `3>1` causes a swap to `[1,3,2]`. At `i=2,j=1`, `3>2` produces `[1,2,3]`. Then `j=0` tests `1>2`, fails, and stops. The prefix is sorted after each outer iteration.

The complete classic MIPS32 implementation follows. Its 24-byte teaching frame omits the 16-byte o32 outgoing argument home area. It is valid with the supplied leaf `swap` under this declared convention; general o32 interoperability requires an outgoing area at offsets 0..15 and relocated saved slots. Preconditions are `n>=0`, `n` aligned accessible words, and nonwrapping effective addresses. Every control transfer retains its delay slot.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete sorting dataflow and cost (continued)

```
sort:
addiu $sp, $sp, -24
sw    $ra, 20($sp)
sw    $s3, 16($sp)
sw    $s2, 12($sp)
sw    $s1, 8($sp)
sw    $s0, 4($sp)
addu  $s2, $a0, $zero
addu  $s3, $a1, $zero
addu  $s0, $zero, $zero
for1:
slt   $t0, $s0, $s3
```

https://github.com/xinhi6716/com

Complete sorting dataflow and cost (continued)

```
beq   $t0, $zero, exit1
nop
addiu $s1, $s0, -1
for2:
slti  $t0, $s1, 0
bne   $t0, $zero, exit2
nop
sll   $t1, $s1, 2
addu  $t2, $s2, $t1
lw    $t3, 0($t2)
lw    $t4, 4($t2)
slt   $t0, $t4, $t3
```

https://github.com/xinhi6716/com

Complete sorting dataflow and cost (continued)

```
beq   $t0, $zero, exit2
nop
addu  $a0, $s2, $zero
addu  $a1, $s1, $zero
jal   swap
nop
addiu $s1, $s1, -1
j     for2
nop
exit2:
addiu $s0, $s0, 1
j     for1
```


Complete sorting dataflow and cost (continued)

```
nop
exit1:
lw    $s0, 4($sp)
lw    $s1, 8($sp)
lw    $s2, 12($sp)
lw    $s3, 16($sp)
lw    $ra, 20($sp)
addiu $sp, $sp, 24
jr    $ra
nop
```

With `v=0x10010000`, `k=1`, scaling gives 4, so the element addresses are `0x10010004` and `0x10010008`. The leaf loads both words before storing them in exchanged order. It modifies only caller-saved temporaries and needs no frame.

Complete sorting dataflow and cost (continued)

```
swap:
sll  $t1, $a1, 2
addu $t1, $t1, $a0
lw   $t0, 0($t1)
lw   $t2, 4($t1)
sw   $t2, 0($t1)
sw   $t0, 4($t1)
jr   $ra
nop
```

For one steady-state clear-loop iteration, array indexing executes `sll+addu+sw+addiu+slt+bne+nop`, seven instruction positions. Pointer traversal executes `sw+addiu+slt+bne+nop`, five positions. The two saved positions compute scaling and base addition. Prologues, initial guards, and exits are outside this count.

The comparison holds word size, traffic, delay-slot policy, and absence of faults/traps fixed. At `n=100`, the specified bodies execute 700 and 500 instruction positions; initialization and final exit checks are additional. Strength reduction and induction-variable elimination can transform the index version into pointer-like code, so source spelling alone does not establish performance.

Compilation and interpretation

C compilation commonly produces target-specific native code. Java bytecode belongs to a virtual machine. A JVM can interpret, compile just in time, or combine the two.

Interpretation pays dispatch costs. A JIT can specialize hot code from profiles, but warm-up and compilation add startup cost. Integer overflow, alignment, atomic operations, and memory models also differ across language runtimes.

Separate language semantics, runtime implementation, and native ISA.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

ARM comparison

The book's ARM discussion concerns the 32-bit ARM era. Conditional execution, shifted operands, and addressing choices reduce some sequences. ARM and MIPS both illustrate load/store organization and procedure conventions.

Encoding, flags, and extensions differ. Do not transfer a 32-bit ARM example into AArch64 or assume every version has the same operations.

Identify the architecture version before interpreting a conditional instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

x86 comparison

x86 instructions have variable lengths and can contain prefixes, ModR/M, SIB, displacements, and immediates. Boundaries are not fixed at four-byte intervals.

Dense encoding and compatibility add fetch and decode complexity. Implementations can translate instructions into internal micro-operations for superscalar execution. ISA count therefore differs from micro-operation count.

A micro-operation cache can reduce repeated decode work without changing the ISA into MIPS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

x86-64 effective addresses and structure layout

Under the default System V AMD64 alignment without packing, with an allocated row and $0 \le j < 4$, for `struct row { int64_t id; int32_t vals[4]; int64_t sum; };`, the fields occupy offsets 0..7, 8..23, and 24..31. Eight-byte alignment gives a 32-byte structure. With base `0x3000`, `i=5`, and `j=3`,

$$address = 0x3000 + 5(32) + 8 + 3(4) = 0x30b4.$$

Let `%rdx` hold the base, `%rdi` hold `i`, and `%esi` hold `j`:

```
movslq %esi, %rsi
salq   $5, %rdi
leaq   8(%rdi,%rsi,4), %rax
leaq   (%rdx,%rax), %rax
```

The first `lea` produces offset `0xb4`; the second adds the base. `lea` evaluates an address expression without reading memory. The scale field admits 1,2,4,8, so a 32-byte structure stride requires the separate shift. Variable instruction length changes layout and decoding; it does not change the address arithmetic. Source-level signed bounds and object validity remain necessary even when the machine can form a bit pattern.

ISA fallacies

A powerful instruction can have long latency or restrict parallel issue. Fewer instructions need not mean fewer cycles. More registers do not eliminate register pressure, calling costs, or spills.

Pseudo-instructions are assembler facilities. Their expansions can depend on operands and immediate ranges. Source lines, architectural instructions, and cycles count different things.

Trace one expression through all three counts.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

ISA design choices

Encoding, compiler support, exceptions, translation, synchronization, and compatibility constrain ISA design. Fixed fields simplify extraction but require multistep constants. Variable formats can improve density while complicating boundaries.

Correctness requires the specified architectural effect. Performance depends on instruction mix, CPI, frequency, and memory paths.

State width, alignment, and extension for every address and condition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical perspective

ISA history contains both regular fixed-field designs and complex encodings. RISC emphasized regular operations suited to compilers and pipelines. CISC often emphasized density and compatibility.

Either family can use internal micro-operations. Historical constraints explain decisions more reliably than timeless claims that a label always wins.

Compare code size, decode, traffic, issue, and compatibility separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter review

- ISA version, delay slots, and assembler ordering determine control flow.
- Addresses use a byte offset; check element size, bounds, and alignment.
- Signed-overflow traps depend on the instruction; the ABI defines preservation and layout.
- LL/SC success, memory ordering, and fairness are distinct guarantees.

References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 2. CSO course slides, Chapter 2.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study